

Technical Assessment: QA Engineer

Claude-first UI automation for a live e-commerce web application

Welcome

Thank you for your interest in the QA Engineer role. [BusPlanner](#) builds the software that school districts across North America use to plan and manage K-12 student transportation, and reliable, well-tested software is at the heart of what we do.

This take-home is the first practical step in the process. It is designed to be realistic rather than exhaustive, and to give you a fair chance to show how you work. Below you will find what we expect and how we would like you to work, followed by the specifics of the exercise itself. Please read the expectations first; they matter to us as much as the task does.

What this exercise is

This is a Claude-first exercise. You will use Claude (or the AI coding agent you normally work with) to stand up a small Playwright automation framework in JavaScript against a live, hosted practice web application, automate its key journeys, add a few negative cases, and leave behind reusable assets and a short process write-up. There is nothing to install or host; you automate against a public URL we provide.

We are not looking for proof that you can hand-type a Playwright suite from memory. We want to see you put the AI to work to the fullest: to scaffold the framework, generate and run tests, investigate the app, iterate on failures, and leave behind a process a teammate could pick up. How you direct, verify, and steer the assistant is a central part of what we assess.

How we want you to work

Keep your judgement in the driver's seat. We still expect durable locators, meaningful assertions, and sound test design, delivered through how well you direct the tool. Specifically:

- **Drive with Claude, don't just autocomplete.** Use it to scaffold, generate, run, and refine. We are interested in the leverage you get, not your typing speed.
- **Build reusable assets around the AI.** Examples: a project instruction file (such as a CLAUDE.md), custom skills, slash commands or sub-agents, helper scripts, or a connector that lets the assistant drive or inspect the app. Leave behind more than a one-off chat.
- **Make it repeatable.** Set things up so a teammate could have the assistant re-run, maintain, and extend your suite without you in the room.

- **Verify everything.** Check what the assistant produces, correct it, and be ready to explain why each choice is right. Output you cannot defend counts against you.

Time

Please aim for up to about two hours. This is intentionally scoped to be modest. If you run short, prioritise the featured checkout scenario and showing your AI workflow clearly.

Using AI tools

Using AI is expected and encouraged; this exercise is built around it. Please keep a log of how you directed the assistant, and be ready to talk through what you did and how you checked the output when we meet. In the review we will ask you to extend your solution live by directing the assistant, so build in a way you can keep working in.

The application under test

You will automate against a live, hosted practice application. There is nothing to install or run locally.

Application: Practice Software Testing (Toolshop), an online tool store.

URL: <https://practicesoftwaretesting.com>

Test accounts (shared public demo, password welcome01 for both):

Role	Email	Password
Customer	customer@practicesoftwaretesting.com	welcome01
Admin	admin@practicesoftwaretesting.com	welcome01

This is a shared public demo. Its data resets periodically and may be changed by other users, so use unique values for anything you create and do not rely on data persisting between runs.

Deliverables

Please produce the following. Each builds on the same project.

1. The automation capability

A Playwright and JavaScript project, plus the assets you built around Claude to produce and maintain it. We are interested in how it is structured (test organisation,

reuse through page objects or fixtures, configuration), how someone runs it, and what reusable AI assets you left behind. Include short run instructions in your README. Aim for something a real team could pick up and keep using.

2. Automated test flows

Automate these core journeys, choosing meaningful assertions on real outcomes rather than just that buttons respond:

- Sign in with valid and with invalid credentials.
- Find products through search and through a filter or sort.
- Add a product to the cart and verify the cart contents.
- Progress through checkout as far as is reasonable.

3. Featured scenario (required): verified multi-item checkout

This is the scenario we most want to see automated well. Build it as one robust end-to-end test, or a small focused set:

1. Sign in as the customer and add at least two different products to the cart from their product pages.
2. In the cart, change the quantity of one item and remove another, then verify the totals: each line total should equal unit price times quantity, and the cart total should equal the sum of the line totals. **Read these values from the page and assert the arithmetic; do not hard-code amounts.**
3. Proceed through checkout. At the payment step, choose a payment method and verify that the method-specific fields appear before you complete the order.
4. Complete the order and assert the confirmation.

4. Negative testing and observations

Add a few negative or edge cases, for example a data-driven set of invalid logins, empty or invalid checkout fields, or quantity edge cases such as zero or a very large number.

While you work, keep a short notes file (**OBSERVATIONS.md**) capturing anything you consider a defect, a validation gap, inconsistent behaviour, or a testability or risk concern, with steps to reproduce and why it matters. There is no fixed list to find; we are interested in what you notice and how you judge it.

5. Your process and handoff

Show us how you worked with the AI. Include both of the following:

- **A process or session log** - your prompts and the key turns of how you directed the assistant (an exported transcript, a PROMPTS.md, or similar). We want to see how you steered and corrected it, not a polished after-the-fact story.
- **A short HANDOFF.md** - how a teammate would have Claude maintain and extend this suite: where the instructions and assets live, how to add a new test, and what you would automate next.

What we are evaluating

- **Claude-first orchestration and leverage** - how fully and effectively you put the AI to work: the reusable assets you built, the workflow you set up, the coverage and quality you achieved through it, and how well you verified its output.
- **Test design and coverage** of the key journeys, with meaningful assertions on real outcomes.
- **Selector resilience** - durable, user-facing locators you can justify. The app exposes stable data-test hooks; how you choose and reason about locators is part of what we assess.
- **Framework quality** - structure, reuse, and clean, idiomatic Playwright.
- **Edge-case thinking** and the quality of your written observations.
- **Communication** - how clearly you explain and defend your choices, including your AI use.

Submitting your work

When you are done, zip your project and include:

- Your Playwright project (source, config, and a README with run instructions), excluding node_modules.
- The reusable AI assets you built (for example a CLAUDE.md, skills, commands, sub-agents, connectors, or helper scripts).
- Your negative tests and your OBSERVATIONS.md file.
- Your process or session log and your HANDOFF.md.

Please return your submission within 48 hours of receiving this case by emailing to ahnaf.rahman@busplanner.com and yusra.abbasi@busplanner.com.

What happens next

If you qualify upon your submission, we will set up a 45-minute call to review the solution together. We will ask you to extend it live by directing the assistant, so build in a way you can keep working in.

We are looking forward to seeing how you work!